PRACTICAL NO. 7

AIM: A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint) The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below)

Process	Allocation Printer	Allocation ROM	Allocation Hard Disk	Allocation RAM	Max Printer	Max ROM	Max Hard Disk	Max RAM
Google Drive	0	0	1	4	0	6	5	6
Firefox	0	6	3	2	0	6	5	2
Word Processor	0	0	1	2	0	0	1	2
Excel	1	0	0	0	1	7	5	0
PowerPoint	1	3	5	4	2	3	5	6

Available Resources			
Printer	**ROM**	**Hard Disk**	**RAM**
1	6	2	0

Write a program to:

- Calculate current need of resources by each process Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

CODE:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
 int n = 5;
 int m = 4;
```

```
char* processes[] = {"Google Drive", "Firefox", "Word Processor", "Excel", "PowerPoint"};


int alloc[5][4] = {
   {0, 0, 1, 4},
   {0, 6, 3, 2},
   {0, 0, 1, 2},
   {1, 0, 0, 0},
   {1, 3, 5, 4}
};


int max[5][4] = {
   {0, 6, 5, 6},
   {0, 6, 5, 2},
   {0, 0, 1, 2},
   {1, 7, 5, 0},
   {2, 3, 5, 6}
};


int avail[4] = {1, 6, 2, 0};

int need[5][4];
printf("--- Need Matrix ---\n");
for (int i = 0; i < n; i++) {
   printf("%-15s: ", processes[i]);
   for (int j = 0; j < m; j++) {
      need[i][j] = max[i][j] - alloc[i][j];
      printf("%d ", need[i][j]);
   }
   printf("\n");
}
printf("\n");


bool finish[5] = {false, false, false, false, false};
int safeSeq[5];
int work[4];
for (int i = 0; i < m; i++) {
   work[i] = avail[i];
}
```

```
int count = 0;
while (count < n) {
   bool found = false;
   for (int p = 0; p < n; p++) {
      if (finish[p] == false) {
         int j;
         for (j = 0; j < m; j++) {
            if (need[p][j] > work[j]) {
               break;
            }
         }

         if (j == m) {
            for (int k = 0; k < m; k++) {
               work[k] += alloc[p][k];
            }
            safeSeq[count++] = p;
            finish[p] = true;
            found = true;
         }
      }
   }


   if (found == false) {
      printf("System is NOT in a safe state! Deadlock detected.\n");
      return 0;
   }
}


printf("System is in a SAFE state.\n");
printf("Safe Execution Sequence: \n");
for (int i = 0; i < n - 1; i++) {
   printf("%s -> ", processes[safeSeq[i]]);
}
printf("%s\n", processes[safeSeq[n - 1]]);

return 0;
}
```

OUTPUT:

```
hari_ai@Harryfunlapcare: ~
hari_ai@Harryfunlapcare:~$ nano bankers.c
hari_ai@Harryfunlapcare:~$ gcc bankers.c -o bankers
hari_ai@Harryfunlapcare:~$ ./bankers
--- Need Matrix ---
Google Drive   : 0 6 4 2
Firefox        : 0 0 2 0
Word Processor : 0 0 0 0
Excel          : 0 7 5 0
PowerPoint     : 1 0 0 2

System is in a SAFE state.
Safe Execution Sequence:
Firefox -> Word Processor -> Excel -> PowerPoint -> Google Drive
hari_ai@Harryfunlapcare:~$
```